ASIC Implementation of a Systolic Array Based DNN Accelerator

Project I (BECE497J) – Review Presentation

Vaibhav [Reg. No.] [Team member 2] [Team member 3]
Guide: Dr. Ravi S — Vellore Institute of Technology — October 2026

Contents

1. The project in simple words
2. Research domain & problem
3. Literature survey & research gap
4. Objectives & novelty
5. System overview (sensor $\rightarrow$ host $\rightarrow$ chip)
6. The neural network used
7. Software / Python flow
8. Hardware architecture & data movement
9. Methodology & analytical model
10. Implementation status & tools
11. Results: simulation, synthesis, LEC
12. Tabulation, SDG alignment
13. Timeline for remaining work
14. Conclusion & references

The project in simple words

What problem?

Recognising something in a picture (here, a handwritten digit) needs millions of multiply–add operations. A normal processor does them slowly and uses a lot of energy.

What is our idea?

Build a **small dedicated chip** whose only job is these multiply–adds: a 4×4 grid of 16 tiny calculators working in a rhythm (a *systolic array*).

What have we built so far?

The complete chip design in Verilog (15 files), the Python tools that train and shrink the neural network, a testbench that proves the chip gives the right answers, and the synthesis + equivalence-check results.

Normal processor
one calculation at a time



4×4 grid of multiply–add units

Our accelerator
16 calculations every clock

In one line: *we trained a digit-recognising network in Python and designed an ASIC that can run it.*

Research domain & problem focus

Research domain

VLSI design — **hardware accelerators for deep neural network (DNN) inference** at the edge.

Problem

- DNN layers are dominated by **matrix multiplication** (multiply–accumulate).
- On a general CPU these run serially, and fetching data costs more energy than the arithmetic itself.
- Edge devices (sensors, wearables) need fast, low-power, *fixed-function* inference.

Problem focus of this work

Design, verify and implement (RTL $\rightarrow$ GDSII) a **compact, programmable INT8 systolic-array accelerator** that runs a trained network layer by layer, using on-chip data reuse to cut memory traffic, and prove it is **bit-exact** against a Python reference.

Why INT8?

8-bit integer multipliers are much smaller and cheaper than 32-bit floating point, and for our network cost **no accuracy** (96.94% in float, INT8 Python and on the RTL).

Literature review & research gap identification

Architecture	Dataflow & precision	Memory reuse	Key limitation in edge AI
Google TPU v1 (ISCA 2017)	Weight-stationary 256×256, INT8 / 32-bit acc	Unified Buffer (SRAM)	Host CISC overhead; low utilisation at small batches (M=1..8)
Eyeriss (ISSCC 2016)	Row-stationary 14×12, 16-bit fixed point	Global buffer + register files	Complex control; large area for local register files
DianNao (ASPLOS 2014)	Central NFU engine, 16-bit fixed point	Multi-banked NBin / SB	Routing capacitance; memory-bus contention at scale
Generic edge systolic	Output-stationary, INT8 / FP16	Streamed from SRAM	Re-reads activations; no integrated requantization
Proposed (this work)	**Weight-stationary 4×4, signed INT8 / 32-bit acc**	**LRB + ping-pong SRAM**	**Fused PPU; ≈74% fewer activation SRAM reads**

Research gaps addressed by this project

1. **Redundant activation re-reads** — tiled arrays re-fetch the same activations for every column tile ⇒ **Local Reuse Buffer**
2. **Per-layer host scheduling** — CPU reprograms pointers layer by layer ⇒ **autonomous descriptor FSM**
3. **Post-processing round-trips** — bias, scaling, ReLU go back to main memory ⇒ **fused 3-stage PPU**
4. **Sign and overflow pitfalls** — unsigned PEs break signed weights; short accumulators overflow ⇒ **signed INT8, 32-bit accumulation**

Objectives & novelty of the work

Objectives

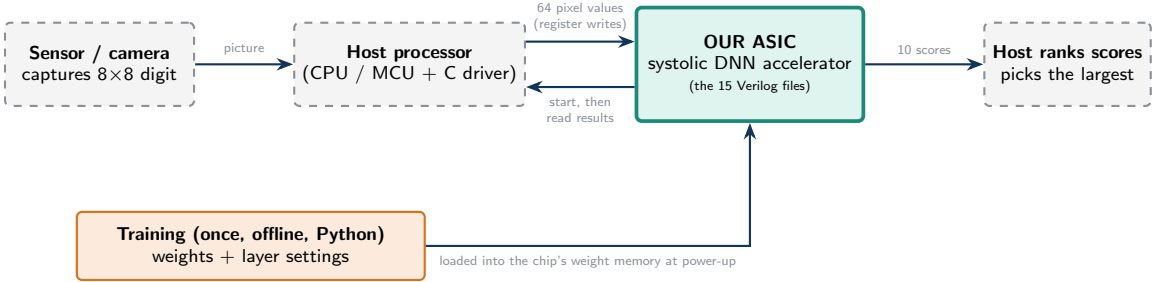
1. Design a weight-stationary INT8 systolic array accelerator in synthesizable Verilog.
2. Make it *programmable*: network = data loaded at run time.
3. Reduce SRAM traffic with a local reuse buffer.
4. Prove correctness against a bit-exact Python model.
5. Synthesize and formally check the netlist; take the design towards GDSII.

Novelty lies in the integration, the reuse buffer and the verified open flow — not in a new algorithm.

What is novel / different

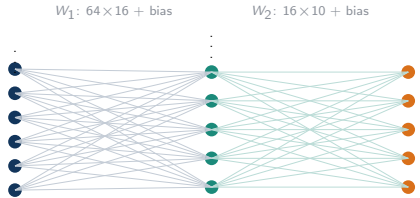
- **Network is data, not hardware**: layer descriptors + weights are loaded into SRAM; same chip runs other dense networks.
- **Local Reuse Buffer (LRB)**: activation SRAM reads per batch fall from **608 to 160** (3.8×); 74% of activation fetches served on-chip.
- **INT8 end to end on chip**: accumulation, bias, rescale and ReLU are done in hardware; the host does no maths between layers.
- **Complete open flow** from PyTorch to register program, verified bit-exact.
- **Host-ready**: register bus + AXI4-Lite bridge, C driver, hardware performance counters.

System overview: how it works in the real world



Grey dashed = not built in this project, *simulated*: sensor → dataset pictures; host → `prog.hex` replayed by the testbench; ranking → `report.py`, `compare_one.py`.
Teal = the deliverable (ASIC). **Orange** = offline Python tools.

The neural network used: digit recognition



Shapes for a batch of 8 pictures: $8\times64 \to 8\times16 \to 8\times10$.
1,184 weights + 26 biases in total.

Dataset

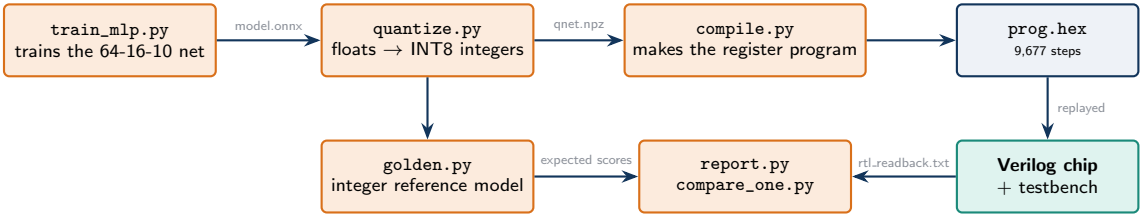
UCI *Optical Recognition of Handwritten Digits* (Alpaydin & Kaynak, 1998), loaded through scikit-learn. 1,797 images of 8×8 pixels.

1,437 for training, **360** held back for testing (fixed split, stratified).

How a digit is decided

The chip outputs 10 scores; the **largest score is the predicted digit**. Example: $[-12, 4, -7, \mathbf{91}, -30, \ldots] \Rightarrow 3$.

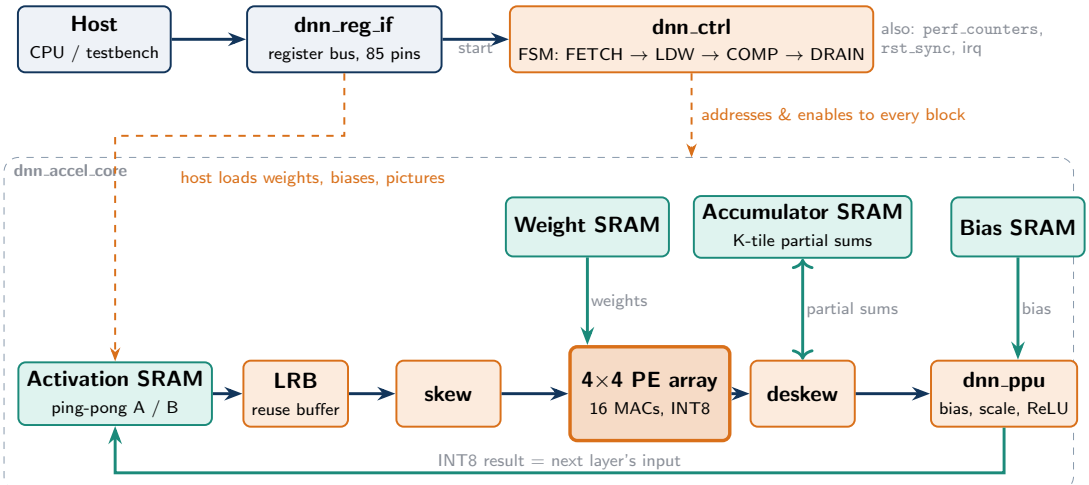
Software flow: the Python scripts and what they do



The chip's scores must equal the Python scores, number for number.

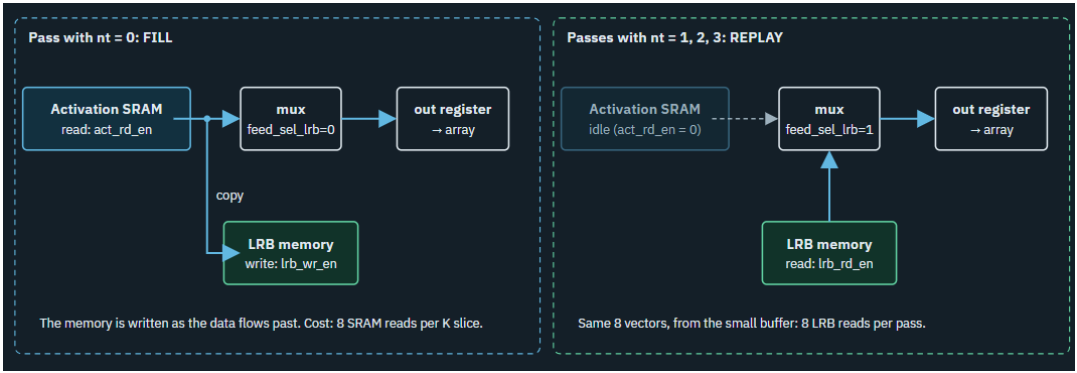
Script	**Job (in simple words)**
`train_mlp.py`	Trains the network on 1,437 pictures (PyTorch); saves the weights (ONNX).
`quantize.py`	Converts decimal weights into small 8-bit integers + works out the multiply/shift numbers.
`compile.py`	Acts like the host driver: writes the list of register writes that loads weights, sends pictures, starts the chip, reads scores.
`golden.py`	Same arithmetic in plain integer Python — the "answer key" the chip must match exactly.
`report.py`, `compare_one.py`	Read the chip's output, pick the biggest score per picture, compute accuracy; show chip vs Python side by side.
`show_digits.py`	Displays the actual test pictures in a browser.

Functional block diagram of the accelerator



Reading it: data flows left to right along the bottom row (pictures → array → result); weights, partial sums and biases join from the memories above; the controller sequences everything. 15 Verilog files in total.

Local Reuse Buffer (LRB): why it exists and how it works

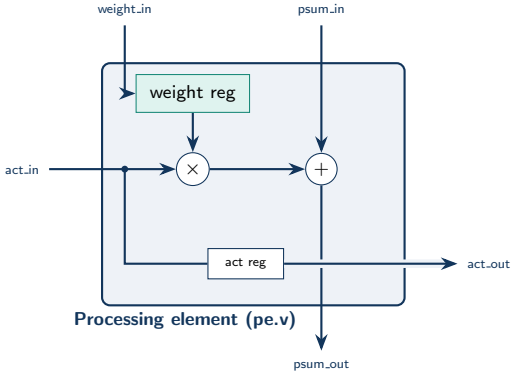


Problem. A layer is cut into tiles; every column tile (nt) needs the *same* activations again.

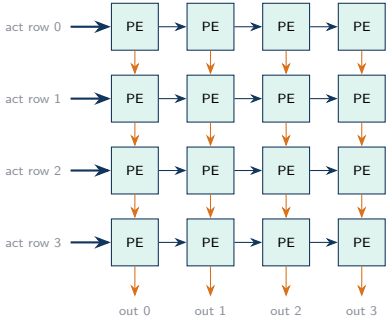
Idea. First tile: read the SRAM and copy to the LRB. Other tiles: replay from the small LRB; the SRAM stays idle.

Result. SRAM reads per batch: **608** $\rightarrow$ **160** (-73.7%); measured = analytical $(NT-1)/NT$.

The processing element and the systolic array



Signed INT8 $\times$ INT8 $\rightarrow$ 16-bit product, summed in a 32-bit accumulator.



$$\texttt{psum_out} \leftarrow \texttt{psum_in} + \texttt{weight} \times \texttt{act_in}$$

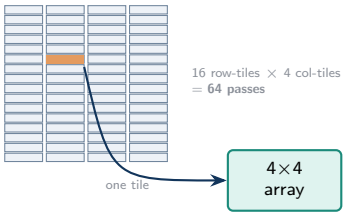
$$\texttt{act_out} \leftarrow \texttt{act_in}$$

$\rightarrow$ activations move right, $\downarrow$ partial sums move down.
Weights stay in place (**weight-stationary**).
Each column adds 4 products = one dot product.

How data moves: one batch through the chip

1. **Tile:** the 64×16 weight matrix is cut into 4×4 tiles (64 for layer 1, 12 for layer 2).
2. **Load** one tile's weights down the columns; they stay put.
3. **Stream** 8 pictures through; *skew* delays row r by r clocks so each activation meets the right partial sum.
4. **Reuse:** the first pass fills the *LRB*; later passes replay from it instead of re-reading SRAM.
5. **Accumulate** the partial sums of all tiles in the accumulator SRAM.
6. **Post-process (PPU):** bias, scale, round, ReLU, clamp to INT8.
7. **Ping-pong:** layer 1's output is written to the other half of the activation memory = layer 2's input.

Layer-1 weights W_1 (64×16) as 4×4 tiles



Per batch of 8 pictures	
Layer 1 passes	64
Layer 2 passes	12
Total passes	**76**
Clock cycles (measured)	1,771
Cycles per picture	221

Algorithm and analytical model

1. Layer maths (what the array computes)

$$y_n = \sum_k x_k \, w_{k,n} + b_n$$

2. INT8 model (what the chip implements)

$$acc_n = \textstyle\sum_k x_k^q \, w_{k,n}^q \quad \text{(32-bit)}$$

$$y_n^q = \mathrm{clamp}\Big(\big((acc_n + b_n^q) \cdot m + 2^{s-1} \big) \gg s,\ lo,\ 127 \Big)$$

m = multiplier, s = shift, $lo = 0$ with ReLU else -128. No floating point anywhere, so any mismatch with the RTL is a real bug.

3. Performance model

$$\text{images/s} = \frac{f_{clk} \cdot B}{\text{cycles/batch}} = \frac{100\,\text{MHz} \cdot 8}{1771} \approx 4.5 \times 10^5$$

MACs per picture $= 64{\cdot}16 + 16{\cdot}10 = 1184$; peak $=$ 16 MAC/clock; achieved $\approx$ 5.35 MAC/clock (33% utilisation).

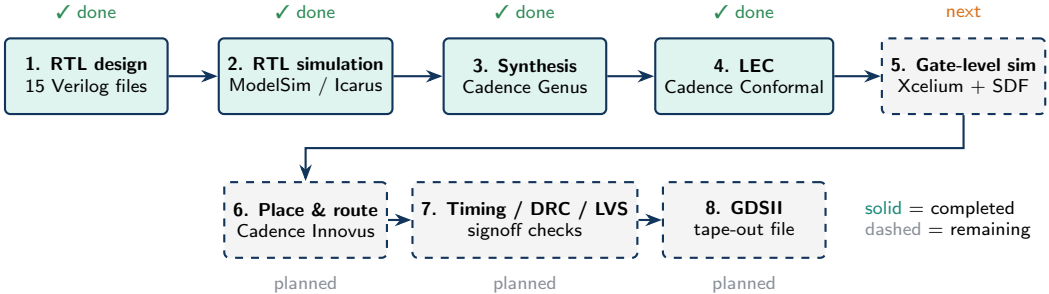
4. No overflow

Needs $\geq 2{\cdot}8 + \lceil \log_2 K \rceil = 16 + 6 = 22$ bits for K=64; we use 32.

5. Validation

Python `golden.py` computes the same integers; the testbench compares every checked read with it: **0 mismatches**.

Methodology: design and verification flow



Implementation details: tools, platforms and status

Stage	Tool / platform	Details	Status
Training	Python, PyTorch, scikit-learn	64–16–10 MLP, UCI digits	✓ done
Quantization & compile	Python (NumPy), ONNX	INT8 weights, `prog.hex` (9,677 steps)	✓ done
RTL design	Verilog-2001, 15 files	Parameterised 4×4 array, register bus + AXI4-Lite	✓ done
Functional simulation	ModelSim (Windows), Icarus, Verilator	0 mismatches, 0 timeouts	✓ done
Driver	C (`driver/`)	Same register stream as the Python compiler	✓ done
Synthesis	Cadence Genus 21.14, typical library, 10 ns clock	75,624 cells, timing met	✓ done
Equivalence check	Cadence Conformal LEC	RTL vs. netlist	✓ done
Gate-level simulation	Cadence Xcelium	Netlist + cell models (+ SDF)	❍ next
Place & route, signoff	Cadence Innovus	Clock tree, routing, STA, DRC/LVS	❍ planned
GDSII	Innovus / streamout	Final layout file	❍ planned

Results (1): functional simulation

INSERT SCREENSHOT HERE

ModelSim console: `*** PLAYER PASSED ***` (`9677 ops, 1313 reads, 0 mismatches, 0 timeouts`)

(replace this box with `\includegraphics{img/sim_console.png}`)

INSERT SCREENSHOT HERE

Single-PE testbench: `tb_pe PASSED` (15 checks)

(replace this box with `\includegraphics{img/sim_pe.png}`)

INSERT SCREENSHOT HERE

ModelSim waveform: `clk`, `reg_wr_en`, `irq`, controller state (LDW → COMP → DRAIN)

(replace this box with `\includegraphics{img/sim_waveform.png}`)

What the output means

- 9677 `ops`: steps replayed from `prog.hex`.
- 0 `mismatches`: every checked read equals the Python reference.
- 0 `timeouts`: the chip always finished.
- `PASSED` = hardware is bit-exact with the model.

Results (2): accuracy and verification

Accuracy on 360 unseen test images

Model	Accuracy
Float (PyTorch)	96.94%
INT8, Python golden	96.94%
INT8, **on the RTL**	**96.94%**

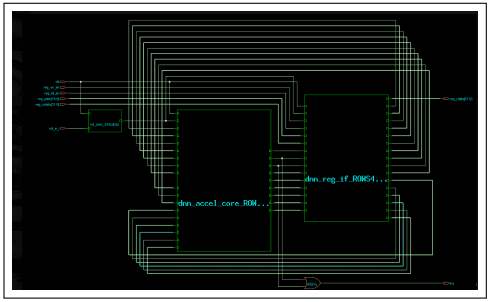
349 of 360 correct

Shrinking to INT8 and running on hardware cost **no accuracy**.

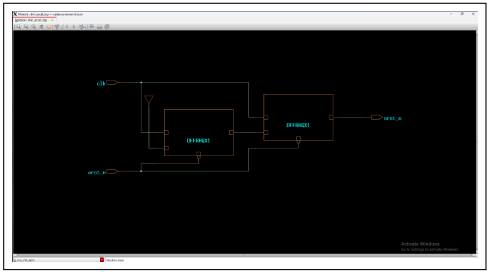
Verification summary

Test	Result
Digit network, 45 batches (1,088 checked reads)	0 mismatches
Single PE testbench (15 checks)	PASS
240 random networks, 7 configurations	ALL PASSED
116 illegal-command tests	ALL PASSED
AXI4-Lite bridge with random stalls	PASS
C driver = Python register stream	PASS
19 injected design faults (mutation test)	all caught

Results (3a): synthesis schematics (Cadence Genus)



Top level: reset synchronizer, `dnn_accel_core`, `dnn_reg_if` and an OR gate that forms `irq`.

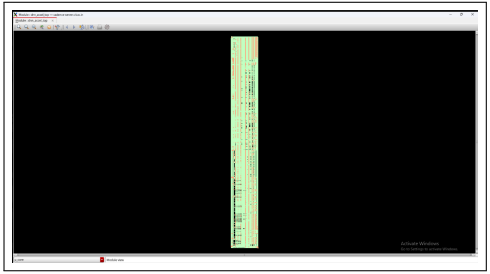


Reset synchronizer: two flip-flops (reset asserts immediately, releases in step with the clock).

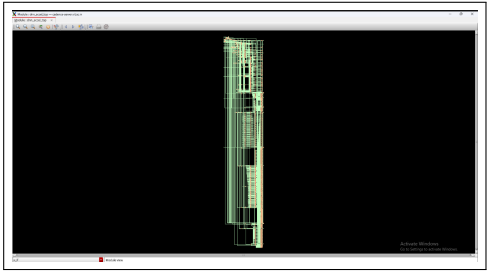
What to notice

- The netlist has the same three blocks as the RTL hierarchy: reset sync, core, register interface.
- The register interface and the core talk through the wires between the two big boxes; the OR gate merges done and error into `irq`.

Results (3b): synthesized blocks (Cadence Genus)



`u_core` **after mapping:** about 74,000 standard cells (flattened); contains the array, memories, controller and PPU.



`u_if`: the register interface (address decode, descriptor and status registers).

What to notice

- `u_core` is flattened: array, memories, controller and PPU are mapped to about 74,000 gates and flip-flops.
- `u_if` is small (about 1,200 cells): address decoding plus the descriptor and status registers.

Results (4): synthesis reports — timing and power

Timing (clock 10 ns = 100 MHz)

Setup slack	**+1,144 ps (MET)**
Data path delay	8,662 ps
Setup time / uncertainty	94 ps / 100 ps
Estimated max. clock	≈ 113 MHz

Critical path: control register → PPU rounding adder → shift → clamp → output register. Conditions: typical, wireload *enclosed*, ideal clock.

Power (no switching-activity file)

Component	mW	Share
Leakage	5.43	7.95%
Internal	60.48	88.53%
Switching	2.40	3.52%
Total	**68.3**	100%

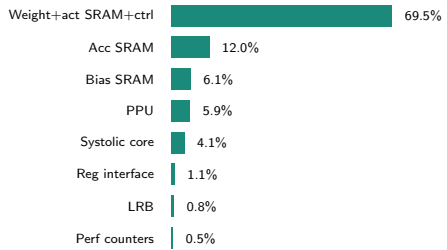
Registers 91.3%, logic 8.7%. Clock-tree power not included (ideal clock).

Pre-layout estimates: wires, clock tree and real switching activity will change the numbers after place and route.

Results (5): synthesis report — area

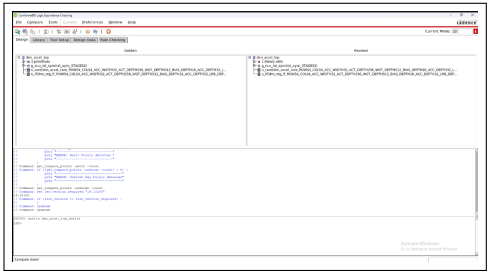
Block	Cells	Area	%
dnn_accel_top	75,624	953,673	100
Weight + act. SRAM + ctrl*	–	663,085	69.5
Accumulator SRAM	5,951	114,452	12.0
Bias SRAM	3,101	58,425	6.1
PPU	6,560	55,825	5.9
Systolic core (array+skew)	4,778	38,804	4.1
Register interface	1,209	10,466	1.1
LRB	616	8,048	0.8
Perf counters	689	4,522	0.5
Reset sync	2	41	≈0

* flattened into `u_core`; obtained as `u_core` minus the listed sub-blocks. Area in library units.

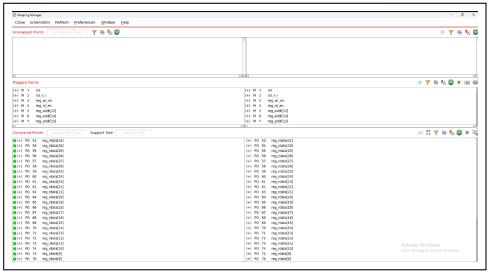


Why so large? The SRAMs are currently built from flip-flops, since no memory macros were used. Real SRAM macros from the foundry would shrink this a lot. Total flip-flops: ≈ 33,657.

Results (6a): logic equivalence check (Cadence Conformal)



Golden (RTL) vs. **Revised** (Genus netlist): same hierarchy; the log ends with "Compare done".

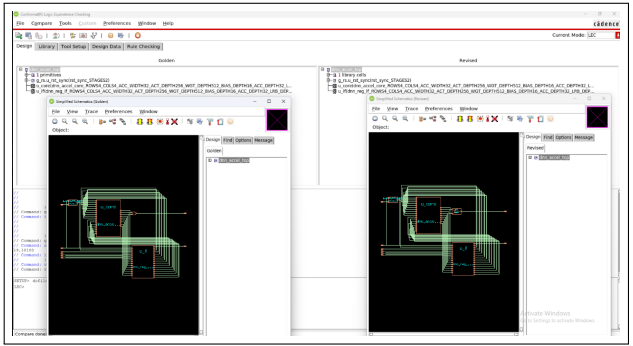


Mapping manager: no unmapped points; the compared points (e.g. `reg_rdata[...]`) are marked equivalent.

How to read it

- **Golden** = our RTL; **Revised** = the gate netlist written by Genus. The tool maps every input, output and flip-flop of one to the other.
- **Mapped points:** inputs such as `clk`, `reg_addr`; **compared points:** outputs and registers whose logic is proven equal.

Results (6b): equivalence check — schematics and meaning



Simplified schematics of the golden (left) and revised (right) designs.

What LEC proves

The synthesized gate netlist is **logically identical** to the RTL for *every possible input* — a mathematical proof, not a test. So synthesis did not change what the design does.

Tabulation: key numbers at a glance

Design / performance	Value
Array / precision	4×4, INT8
Network	64 → 16 → 10
Weights + biases	1,184 + 26
Cycles per batch of 8	1,771
Throughput @ 100 MHz	≈ 452,000 images/s
Compute rate	≈ 1.07 GOPS
Peak MACs / clock	16
Array utilisation	33.4%

Implementation	Value
Verilog files / pins	15 / 85
Std cells (synthesis)	75,624
Area (library units)	953,673
Setup slack @ 10 ns	+1,144 ps
Power (estimate)	68.3 mW
Efficiency (estimate)	≈ 15.7 GOPS/W
Accuracy on RTL	96.94%

Effect of the local reuse buffer (per batch of 8)

	Without LRB	With LRB
Activation SRAM reads	608 words	**160 words** (3.8× fewer)
Activation fetches served by LRB	0%	73.7%

GOPS = 2 operations per MAC × 1,184 MACs × images/s; efficiency = GOPS / power (pre-layout power, no activity file).

Observations and honest limitations

What went well

- Hardware is bit-exact with the Python model; no accuracy lost to INT8.
- Timing is met with margin ($+1.14$ ns at 100 MHz).
- LEC confirms synthesis preserved the logic.
- LRB cuts activation reads by $3.8\times$.

What we will improve

- **Area:** memories are flip-flops $\Rightarrow$ move to SRAM macros.
- **Timing:** critical path is the PPU rounding adder $\Rightarrow$ pipeline / faster adder.
- **Utilisation:** array is busy 33% of the time $\Rightarrow$ overlap load and compute.
- **Power:** re-estimate after clock tree and with real activity.
- **Testability:** scan-chain (DFT) insertion is future work.

Demonstration of working modules

INSERT SCREENSHOT HERE

Demo video / screen recording:
running `run_modelsim.bat` → `PASSED`
(add a link or QR code here)

(replace this box with
`\includegraphics{img/demo_video_link}`)

Suggested live demo (3 steps)

1. `show_digits.py`: show the test pictures.
2. Run the simulation (`run_modelsim.bat`): see `PLAYER PASSED`.
3. `compare_one.py 17`: chip scores and Python scores side by side — identical; biggest score = predicted digit.

Research objective – SDG alignment

UN Sustainable Development Goal	Target	Link to this project
SDG 9 Industry, Innovation & Infrastructure	9.5 Enhance scientific research and upgrade technological capabilities	Indigenous, fully documented AI-accelerator ASIC design flow; builds chip-design capability.
SDG 7 Affordable & Clean Energy	7.3 Double the global rate of improvement in energy efficiency	Dedicated INT8 hardware with data reuse uses far less energy per inference than a general processor.

Please confirm the exact SDGs/targets your department expects before the review.

Timeline plan: remaining milestones



Milestone	Deliverable	Target
Gate-level simulation	Netlist passes `tb_dnn_player` (0 mismatches)	late Oct
Netlist improvements	Memory macros, PPU timing fix (optional)	early Nov
Place & route	Clock tree, routing, post-layout timing/power	early Nov
Signoff	Timing, DRC, LVS clean	mid Nov
Final deliverables	GDSII, project report, presentation	end Nov

Dates are proposed; adjust to the department's schedule.

Conclusion

Summary

- A compact, programmable INT8 systolic-array DNN accelerator is designed in 15 Verilog files.
- It runs a digit-recognition network at **96.94%** accuracy, bit-exact with the Python reference.
- Synthesis: 75,624 cells, **timing met** at 100 MHz, $\approx$ 68 mW (pre-layout).
- **LEC** confirms the netlist matches the RTL.

Next

- Gate-level simulation
- Place & route with Innovus
- Signoff checks and GDSII
- Optional: SRAM macros, DFT, PPU timing

References

1. H. T. Kung, "Why systolic architectures?", *IEEE Computer*, vol. 15, no. 1, 1982.
2. N. P. Jouppi et al., "In-datacenter performance analysis of a Tensor Processing Unit", *ISCA*, 2017.
3. Y.-H. Chen, T. Krishna, J. Emer, V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks", *IEEE JSSC*, 2017.
4. V. Sze, Y.-H. Chen, T.-J. Yang, J. Emer, "Efficient processing of deep neural networks: A tutorial and survey", *Proc. IEEE*, 2017.
5. B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference", *CVPR*, 2018.
6. H. Genc et al., "Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration", *DAC*, 2021.
7. E. Alpaydin, C. Kaynak, "Optical Recognition of Handwritten Digits", UCI Machine Learning Repository, 1998. DOI 10.24432/C50P49.
8. Cadence Genus, Conformal LEC, Innovus and Xcelium user documentation.

Thank you

Questions?